



Network Architecture Lab - Semester Project

Project Final Report

Author:

Sheng-Wen Chen [414932]

June 2026

Contents

1. Introduction

- Token Bucket Filter (TBF) is used by ISPs to enforce traffic limits, regulating traffic and clearing up network congestion. There can be two ways of assigning TBF, one is where multiple users share the same TBF for the same application (such as streaming Youtube), and the other is each user is assigned one TBF. Under the case that the client is assigned one TBF, it is possible for the sender to optimize the queue size, enabling the application to send in a rate that optimizes the full queue, sending in a rate (cwnd) that does not cause traffic policing (dropping the packets when the queue is full). - While there are works (Franzisk's work) exploring algorithms to estimate this cwnd, and works to estimate the traffic policing rate (Martyna's), both experiments were conducted in a ns3 environment. - Goal of this project is to integrate the works together to see whether the algorithm works in physical scenario. In addition

2. Background

2.1.

2.2. Existing work

- Franzisk's work: algorithm 1 and algorithm 2 => algorithm1: $\text{window} = \text{BDP} + \text{buffer size} - 1 \text{ MSS}$
=> $\text{window} = \text{BDP} + 1 \text{ MSS}$

- Martynas work: policing rate => we focus on the client side cumulative approach, where "when the queue size" is small and the burst size is not that big, the cumulative approach could accurately estimate the traffic policing rate => essentially

3. Experimental Setup

The first challenge was to extract the cwnd

3.1. Using WeHe as a replay device and extracting the cwnd using

- WeHe is an application used to detect whether the ISP is doing traffic differentiation and to check whether the bottleneck is on the end point ISP - Wehe could replay a pre-recorded packets between the server and the client, and measure whether there is a throughput difference - In order to implement algorithm 1, we initially decided to use Wehe as a replay server and create another process that runs "ss" script in the background to extract the cwnd of the WeHe server - however, the "ss" command could not keep-up to the pace of fast changing cwnd rate (kernel's overhead to process a Netlink dump and copy that memory back), causing us to not be able to see an ideal time cwnd shark-tooth shaped graph (Figure report/images/physical_setup1/WeHe_youtube.png)

3.2. Extracting the cwnd by adding custom cca in the modules

=> instead of using the "ss" command, we decided to modify the linux kernel code that controls the congestion control algorithm directly, by making the script print `printf("everytime it receives an ACK") => in addition, we decided to build our simple client and server setup, as we could not control the sending rate, and the amount of data how to load our own cca modules? where did I get this info from... we learn how to do this partially from this documentation.. to be however even after setting this we were not able to get the ideal shark-tooth shaped diagram (Figure report/images/physical_setup1/WeHe_youtube.png) Figure shows the case when we tested our server sending 20 Gb to the client, as it can be seen the slow start did not start from cwnd`

3.3. Cleaning up the logs made by the cca module (linux uses ring buffer)

- The cause of the previous issue was due to the ring buffer - Everytime `prinfo` is called, the log will be written to... which is a ring buffer. Since we were reprinting `prinfo` every time we receive an ACK, the ring would be filled up quickly and overwrite the existing logs. A "cleaning process" was required - Thus we defined a backend process that would be called when the server starts running, which in addition, we modified the saving rate from "every ACK" to "whenever the phase changes" "every 100 ACKs", also added sequential logging.

3.4. Final Base Program Setup

With all of the try and error from before, we then came up with a final experiment base setup as shown below: (will add the design diagram later)

Note: look at `client_server`, my explanation below may be on the old version or unclear. Modify it so that it matches with the new version. Also some explanations may not be very clear, so modify it to be more specific - `run.sh => set the srcport, srcip, CWNDO, "reno" will be used`. The TBF parameters would also be specified here. - when `run.sh` is executed, the script will firstly check if the listener output queue `=> client` which was not ideal the script will then start the server, which the server would start 3 sub processes:
- subprocess 1: log extractor `=> extract the logs from ring buffer, search the ones from prinfo and that contain the port number`
- subprocess 2: log cleaner `=> clean the ring buffer` - subprocess 3: rarrival extractor `=> used to determine the Rarrival rate. Note that the server does a BULK send, that is it tries to send the data as fast as possible, capped at 1Gib - the server then waits until the server finishes sending all data it would then calculate the loss rate, throughput (after the slow start phase), store them in a graph containing the first 500 prinfo logs`

3.5. Physical Setup

- server and client connected to the same router (within the same subnet) - so there is no random bottleneck in between the client and the server - the only bottleneck that we will have is the qdisc TBF set in front of the server

4. Experiment 1: testing Franziska approach

4.1. parameters

$QUEUE_SIZES = (6250, 12500, 25000, 50000, 100000)$ $TOKEN_GENERATION_RATES = (2505007501000)$ $BURST_SIZES = (5840125002500050000100000)$

4.2. Changes to the original cca script

- compare "`client_server/custom_cca/my_cca.c`" and "https://github.com/torvalds/linux/blob/master/net/ipv4/tcp_cong.c" and write briefly with small code snippets of how we cap the cwnd

4.3. Finding the empirical values: algorithms

- How do we do it? `=> automation script that does this for us`. Initially, the automation script will run normal TCP New Reno, records the cwnd (our cca works on top of TCP Reno, we just added `prinfo` to the suitable location), and calculate the average cwnd (mountained edge of the shark-tooth graph) `=> generally this average cwnd is "very close" to the empirical cwnd, we thus decreased the cwnd cap by 1, re-run the server client code` the "loss rate" could be set by the user when running the server script (for queue sizes to 12500 we are using loss rate of 0.20 and for queue sizes from 25000 we are using loss rate of 0.05)

4.4. different burst sizes

From the experiments above, we have seen that the assumption holds, where when the queue size is small, empirical cwnd indeed

4.5. Cause of the inaccurate estimation

- $R_{arrival}$ rate is not taken into account – algorithm 2 defined the calculation of the optimal cwnd as $BDP + queue_size - 1MSS$, however, when the packet from the tcp application arrives faster than the token generation rate, it will accumulate in the queue. Q_p penalty – how many bytes accumulate in the queue in each RTT . Q_e effective – how many bytes can the queue still accumulate [11 : 42]. Rather the problem that was, when we set the cwnd so big, TCP was sending more data than the receiver could handle. - the current setup allows the sender to send set amount of data in BULK and "as fast as possible capped in 1Gib" meaning that the enqueuing rate is generally faster than the token generation rate - Taking this into account,

4.6. burst size 50000 to test the penalty approach

=> because in Frabziska's report it stated that "increasing the burst size does not influence throughput under the optimal cwnd algorithms" => burst size 50000 bytes => How did we do this?: reverse calculate using the empirical value

4.7. clamping rwnd from the client side

=> now we know the empirical rate, since we want to establish this cwnd capped on the server side from the TCP handshake phase, we disabled the cwnd cap on the server side to test whether we could still achieve the empirical throughput

5. Experiment 2: Estimation of the rwnd from the client side

Since we know that it is possible to limit the cwnd by setting the rwnd from the client side, we would also like to see whether it is possible for us to estimate this rwnd from the client side (before we were testing the throughput from the server side and manually trying to find out the empirical value)

5.1. Using WeHe as a replay server again

With the Base Setup, it is now possible for us to extract the correct cwnd however, with the token generation that we currently have, the WeHe replay rate cannot really fill up the TBF queue

5.2. Cumulative bytes arrival

Can estimate the BDP when we have balanced queue size burst, however only BDP, we cannot estimate the empirical rwnd because the $R_{arrival}$ rate is not taken into account => the setup : 1, 2ms (so 3 RTTs) of replays where the server

5.3. Receiver-Assisted TBF Enqueue-Rate Estimation

6. Limitations

=> cannot set the RTT in the physical setup (the router could only set queue sizes... does not have OS that could run qdisc)

7. Conclusions